Real-Time Fraud Detection

Matthew Carey

DATA 789 · Project 2 Technical Report

Part A Streaming pipeline and feature store

This project uses simulated purchases to demonstrate how a payment service could flag a potentially fraudulent transaction. The model considers the purchase alongside the customer's recent activity, including how many purchases they made in the past 24 hours and how much they spent per purchase on average.

A.1 System architecture

The simulator in Figure 1 generates purchases and sends them to Kafka, which records them as they arrive. The feature processor reads each purchase and uses its timestamp to look back over the customer's last 24 hours of activity. It counts how many purchases the customer made during that period and calculates the average amount spent per purchase. It saves those two values in Redis under the customer's ID so the prediction API can retrieve them in a single lookup.

A client requests a prediction by sending the purchase details to the API, built with FastAPI. The API uses the customer ID from the request to retrieve the spending summary from Redis. If Redis has no entry for that customer, the API uses zero for both the count and the average. It passes those values and the purchase details to the trained model, then returns the estimated fraud probability to the client along with a flag indicating whether the model classified the purchase as fraudulent.

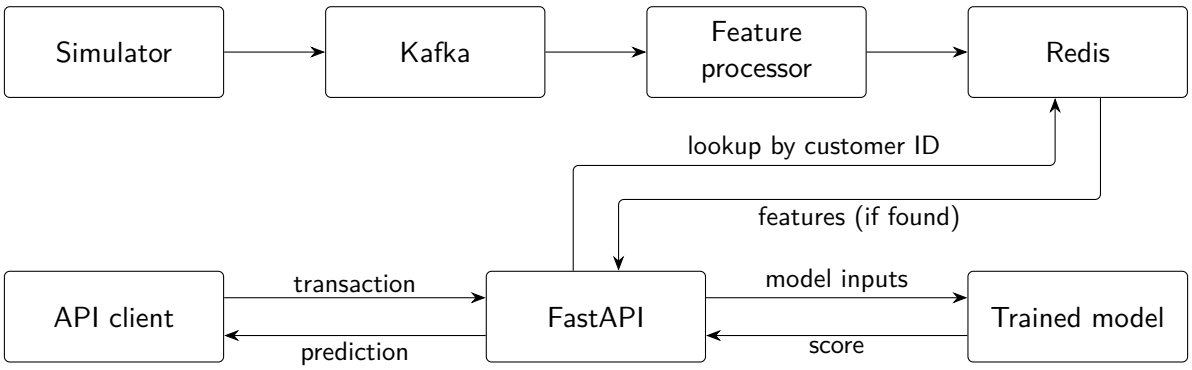


Figure 1: Fraud detection system architecture.

A.2 Topic and partition design

The simulator publishes each transaction to the `transactions` topic. Kafka appends the message to one of the topic's three partitions, preserving the order in which messages arrive within that partition. The feature processor subscribes to the topic and reads the transactions recorded there.

The simulator uses the customer ID as the message key because the feature processor maintains a separate spending summary for each customer. The producer hashes that ID to select a partition, so every purchase by the same customer goes to the same log. One feature processor currently reads all three partitions, but the work could be shared by assigning different partitions to different processors.

A.3 Windowing approach for the streaming features

The feature processor records each purchase under its `customer_id` and uses the latest purchase timestamp seen for that customer as the end of the rolling window. With the default 24-hour window, a purchase at noon on Tuesday brings in all of that customer's recorded purchases after noon on Monday, up to and including the Tuesday purchase. A purchase at exactly noon on Monday falls outside the window. The processor counts the purchases within the window, averages their amounts, and writes both values to Redis.

If a Tuesday morning purchase arrives after the noon summary has been written, the window still ends at noon. The processor includes the morning purchase when it recalculates the count and average, then writes the updated summary to Redis. A late purchase at or before noon on Monday falls outside that window.

Kafka can deliver the same purchase more than once. The processor keeps a set of transaction IDs it has already recorded and checks each incoming ID before adding the purchase to the customer's history. A repeated ID therefore contributes only once to the count and average. Purchases without an ID are counted as they arrive. The processor keeps both the IDs and the window history in memory for the duration of the run.

A.4 Feature store design and measured retrieval latency

After each transaction, the feature processor writes the customer's 24-hour transaction count and average purchase amount to Redis as a single JSON value under `features:<customer_id>`. The API can then retrieve both features with one lookup. For a batch of customers, the feature store sends their keys in one Redis `MGET` request. Each write resets the key's time to live (TTL), which defaults to 48 hours. A test summary with a two-second TTL was available immediately after the write. Once the TTL expired, the same lookup returned no value.

A local benchmark measured how long the feature store took to retrieve and decode a customer's summary. After 50 warmup reads, the benchmark timed 1,000 sequential lookups for the same customer. The 95th percentile was 0.557 ms, meaning that roughly 95% of the

lookups completed in that time or less. This was below the project's 50 ms retrieval target.

A.5 Links to implementation

- Kafka and Redis configuration: `docker-compose.yml`
- Kafka producer and customer key: `src/streaming/transaction_simulator.py`
- Kafka consumer and window calculations: `src/streaming/feature_processor.py`
- Redis storage and retrieval: `src/streaming/feature_store.py`

Part B Serving and containerization

B.1 API design, endpoints, and Pydantic validation

A client requests a fraud prediction by sending a transaction to `POST /predict`. The Pydantic request schema requires a customer ID, purchase amount, merchant category, online purchase flag, and timestamp. The client can also supply a transaction ID to match the prediction to the original purchase. If a required field is missing, such as the purchase amount, FastAPI returns HTTP 422 before the transaction reaches the model.

For a valid transaction, the API uses the customer ID to retrieve the customer's 24-hour transaction count and average purchase amount from Redis. If Redis has no summary for that customer, both features default to zero. The API combines these features with the submitted transaction and scores it using the trained model, which it loads once at startup. The JSON response includes the estimated fraud probability, the fraud flag, the model version, the elapsed time in milliseconds, and the transaction ID when supplied.

A client with several transactions can send them together to `POST /predict_batch`. The API retrieves the customer summaries in one Redis call and returns one prediction per transaction in the order received. The deployment checks API availability through `GET /health`, which returns HTTP 200 when the API is running, while `GET /model/info` reports the model version.

B.2 Containerization with a multi-stage image and Docker Compose

The Dockerfile builds the application image in two stages. The first stage installs the Python packages listed in `requirements.txt`. The second starts from a slim Python 3.11 image, copies in the installed packages, and adds the application code, model files, and data. When the API container starts, Uvicorn serves the application on port 8000 under the non-root account `appuser`. Docker periodically calls `GET /health` on that port to check that the API responds.

Docker Compose uses the same Dockerfile for the simulator, feature processor, and API, giving each service its own startup command. Running `docker compose up` starts these services alongside Kafka and Redis. The simulator publishes transactions to Kafka, and the feature processor consumes them and writes updated customer summaries to Redis. The API reads those summaries from Redis when a client submits a transaction for scoring.

B.3 Blue-green deployment and traffic switch

Blue and green are separate deployments of the prediction API. Clients send requests to nginx on port 8080, and nginx forwards them to whichever deployment is active. While blue serves traffic, a new version can start in green and receive health checks before it serves its

first client request.

The `switch_traffic.sh` script reads the nginx configuration to identify the standby deployment and calls its `/health` endpoint directly. If the health check continues to fail, the script leaves traffic with the active deployment. Once the check succeeds, the script changes the upstream server in the nginx configuration and issues a graceful reload. New nginx workers route incoming requests to the newly active API while the old workers finish requests already in progress.

The previous deployment stays available for rollback. Running the same script again checks that deployment's health and switches traffic back. Figure 2 shows the switch from green back to blue while the client continued sending prediction requests through nginx.

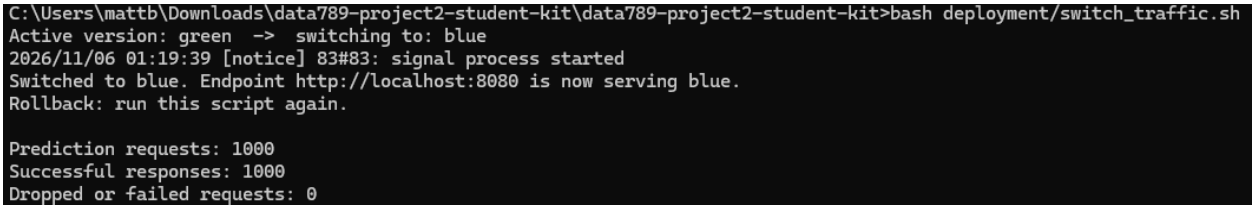


Figure 2: All 1,000 prediction requests sent during this rollback test succeeded.

B.4 Links to implementation

- API endpoints and Pydantic schemas: `src/api/main.py`
- Model loading and prediction: `src/api/fraud_detector.py`
- Image build and runtime health check: `Dockerfile`
- Application services, Kafka, and Redis: `docker-compose.yml`
- Blue and green API deployments: `deployment/docker-compose.blue-green.yml`
- nginx routing: `deployment/nginx/nginx.conf`
- Health check, traffic switch, and rollback: `deployment/switch_traffic.sh`
- Deployment design and switch evidence: `docs/blue_green_design.md`

Part C Performance

C.1 Latency percentiles and throughput

The baseline API retrieves a customer summary from Redis for each prediction. The comparison version lets the API reuse that summary for up to one second. To compare the two, the provided load tester sent 1,000 transactions to `POST /predict` in each trial, waiting for a response before sending the next request. A fixed seed of 789 produced the same sequence of purchases for both versions, and both used the same model and Redis data.

Each of the 20 trial pairs included one run per version. The versions alternated going first, and cached entries had time to expire between runs. All 40,000 requests succeeded. Table 1 summarizes the client-side measurements and the number of reads from Redis.

Table 1: Prediction performance across 20 trials per version. Each cell shows the trial average with the minimum and maximum in parentheses. Percentage changes use the unrounded averages.

Metric	Baseline	One-second cache	Change in avg.
Throughput (requests/s)	441.6 (355.0–471.6)	496.4 (416.6–531.5)	+12.4%
p50 latency (ms)	1.92 (1.80–2.38)	1.67 (1.55–1.98)	−12.7%
p95 latency (ms)	2.60 (2.18–3.92)	2.38 (1.98–3.11)	−8.5%
p99 latency (ms)	3.22 (2.41–5.17)	2.97 (2.24–4.51)	−7.6%
Redis reads per trial	1,000 (1,000–1,000)	319.6 (307–345)	−68.0%

The cached API served about 55 more requests per second on average and had higher throughput in 15 of the 20 pairs. The slowest requests showed more variation: although average p99 latency fell from 3.22 to 2.97 ms, the cached version had a lower p99 in only 12 pairs.

C.2 Bottleneck analysis and optimization

Each trial drew 1,000 transactions from a pool of 200 customer IDs. When two requests for the same customer arrived close together, the baseline API made a second round trip to Redis even if the customer's summary had stayed the same. Timing the steps inside `/predict` showed that retrieving and decoding a summary took 0.263 ms at the median, about as long as scoring the transaction (0.264 ms). Repeated lookups therefore accounted for time that the API could save while still scoring each purchase individually.

The modified API keeps a customer summary in memory for one second after a lookup. A second purchase by that customer within the same second can use the cached count and average amount together with its own transaction details. As Table 1 shows, this reduced Redis reads from 1,000 to about 320 per trial, and average throughput rose 12.4%. The benefit depends on how often a customer returns within that second. The tradeoff is freshness: if the feature processor updates Redis after a summary enters the cache, a prediction can use the previous summary until the entry expires.